Project Overview

Habit Tracker (CLI Backend)
Python-based habit tracking system with SQLite persistence and functional analytics

Core Features:

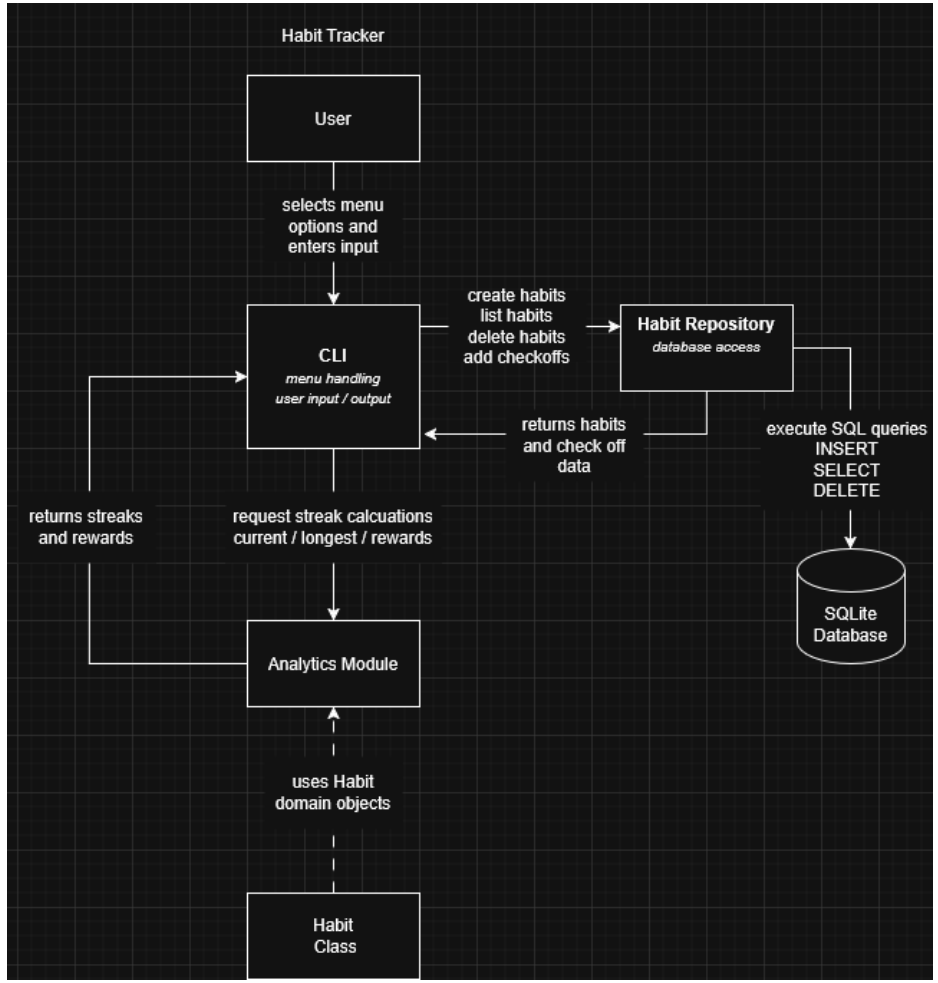
- Create daily & weekly habits
- Record check-offs
- Calculate streaks
- Earn consistency rewards
- Store data between sessions

Architecture Overview

The Application is structured into

Clearly separated modules:

- CLI (User Interface)
- Repository (Database access layer)
- Analytics Module (Functional logic)
- SQLite Database (Persistence)
- Habit Class (Domain Model)



Clear separation of concerns improves maintainability and testability

Habit Class & Data Model

Habit Class

- Id
- Name
- Periodicity (daily/weekly)
- Created_at (timestamp)

Event Log (Checkoffs)

- Habit_id
- Completed_at (timestamp)

The data model and class structure are implemented in a modular and well-documented manner to support maintainability.

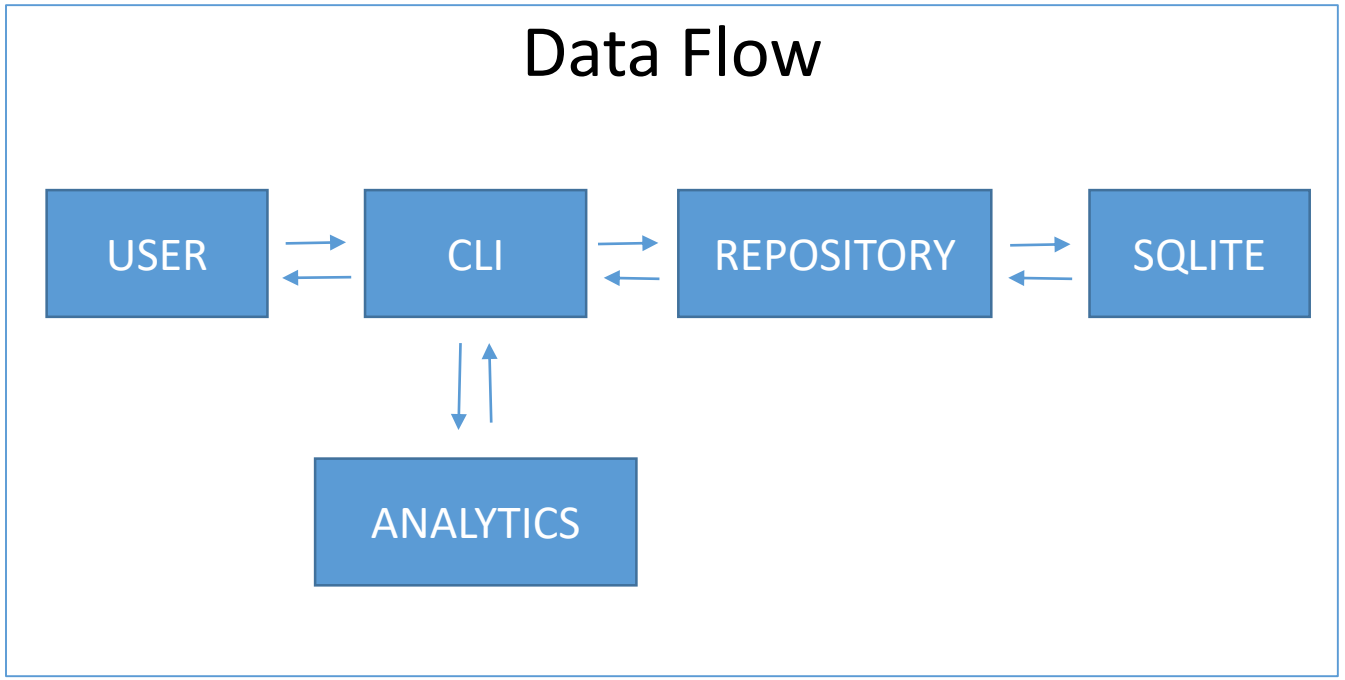
Storage Approach

SQLite Persistence

Why SQLite?

Chosen instead of CSV/JSON to support relational queries and scalability

- Lightweight
- Persistent
- No external dependencies
- Supports relational structure



CLI Workflow

Create → Check-off → Analyze

1. Add a new Habit
2. Check off completion
3. View streaks and rewards

```
=== Main Menu ===
1. Add a new habit
2. Check a habit
3. Show streaks for a habit
4. Show longest streaks
5. Show my rewards
6. Delete a habit
0. Exit
Please select an option:
```

After analytics output, the system pauses for readability.

Analytics (Functional Programming)

Analytics module uses pure functions:

current_streak()

longest_streak_for_habit()

convert_sparkles()

Pure functions that compute results from stored habit and check-off data without modifying the database.

Why functional?

- No database modification
- Predictable results
- Easily testable
- Reusable computation logic

Example:

Daily streak → Consecutive calendar days with atleast one check-off

Weekly streak → Consecutive ISO weeks

Testing Strategy & Early Test Results

Tested Components:

- Analytics calculations
- Repository database operations
- Edge Cases
  - empty data
  - gaps in streaks

Testing Approach:

- In-memory SQLite database
- Dummy checkoff timestamps
- Independent analytics tests

```
tests\test_analytics.py ......                [ 75%]
tests\test_repository.py ..                   [100%]

================ 8 passed in 0.01s ================
```

All tests executed successfully (pytest)

Predefined Habits & 4-Week Test Data

5 Predefined Habits

- Drink Water (daily)
- Brush Teeth (daily)
- Read 10 Pages (daily)
- Gym Workout (weekly)
- Call Family (weekly)

4 weeks of check-off data generated automatically via seed script.

Progress vs. Original Plan

Planned Architecture:

- ➢Modular system design
- ➢Persistent data storage
- ➢Command-line interface
- ➢Analytics module
- ➢Automated testing
- ➢Predefined demo data

Implemented Solution:

- ✓Clean modular architecture
- ✓SQLite database persistence
- ✓Interactive CLI workflow
- ✓Functional analytics module
- ✓Unit testing with pytest
- ✓Automated demo data generation

All planned core components were successfully implemented and validated.

Next Steps Toward Phase 3

Planned Enhancements for Phase 3

- Monthly statistical overview
- Enhanced input validation
- Increased automated test coverage
- Optional GUI or web-based frontend
  - *The modular architecture enables future integration of alternative user interfaces (e.g., GUI or web frontend) without changes to the core logic.*